



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт технологий управления (ИТУ)
Кафедра информационных технологий в государственном управлении (ИТГУ)

ОТЧЕТ ПО ПРОЕКТНО-ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине
«Программирование»

на тему
«Проектно-практическая работа 1»

Выполнили студенты группы - УИБО-06-24	<i>Никитина С.Д., Ильясова К.Ф., Кокунина А.Е.</i>
Проверил	<i>ст. преп. Морозов В.А.</i>

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. УСЛОВИЕ ЗАДАНИЯ.....	5
3. АЛГОРИТМ РЕШЕНИЯ ЗАДАЧИ	9
4. ИНСТРУМЕНТАРИЙ РЕШЕНИЯ ЗАДАЧИ.....	11
5. КОД НА ЯЗЫКЕ C++	12
5.1 Алгоритм реализации веб-интерфейса на HTML и JavaScript.....	12
6. ТЕСТИРОВАНИЕ	13
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17
ПРИЛОЖЕНИЯ	18
Приложение А	19
Приложение Б	22

ВВЕДЕНИЕ

В современных условиях многие производственные и экономические процессы связаны с необходимостью эффективного распределения ограниченных ресурсов. Предприятия постоянно сталкиваются с задачами оптимизации, когда требуется выбрать наилучший вариант производства при наличии ограничений на трудовые ресурсы, материалы или другие производственные факторы.

Одним из подходов к решению подобных задач является использование алгоритмов оптимизации. Среди них особое место занимает метод полного перебора (brute force), который заключается в проверке всех возможных комбинаций решений с последующим выбором наилучшего результата.

Актуальность данной работы заключается в необходимости изучения алгоритмов оптимизации и их программной реализации. Практическое применение подобных алгоритмов позволяет находить оптимальные решения в условиях ограниченных ресурсов.

Целью работы является разработка программного приложения, позволяющего определить оптимальное распределение ресурсов между несколькими продуктами для получения максимальной прибыли.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить постановку задачи оптимального распределения ресурсов;
- разработать архитектуру программного приложения;
- реализовать алгоритм полного перебора;
- разработать пользовательский интерфейс для ввода данных;
- реализовать программу на языке C++;
- провести тестирование разработанного программного решения.

Объектом исследования являются алгоритмы оптимизации распределения ресурсов.

Предметом исследования является программная реализация алгоритма полного перебора для решения задачи оптимального производства.

1. УСЛОВИЕ ЗАДАНИЯ

Согласно условию задания необходимо разработать программное приложение, позволяющее определить оптимальное распределение ограниченных ресурсов между несколькими продуктами.

Пользователь должен иметь возможность задать следующие параметры:

- количество производимых продуктов;
- трудовые затраты на производство одного продукта;
- расход материалов на единицу продукции;
- прибыль от продажи единицы продукции;
- общее количество доступных трудовых часов;
- общее количество доступных материалов.

Основная задача программы заключается в определении такого количества каждого продукта, при котором суммарная прибыль будет максимальной, при условии соблюдения ограничений на ресурсы.

Для решения данной задачи используется алгоритм полного перебора, который рассматривает все возможные комбинации количества продуктов и проверяет выполнение ресурсных ограничений.

В результате работы программы пользователь получает:

- оптимальное количество каждого продукта;
- максимальную прибыль;
- количество проверенных комбинаций;
- время выполнения расчета.

Разработанное приложение реализовано в виде веб-программы, доступной по адресу:

<https://0toxic0.github.io/project-practical/>

2. АРХИТЕКТУРА СИСТЕМЫ

Архитектура системы представлена на рисунке 1

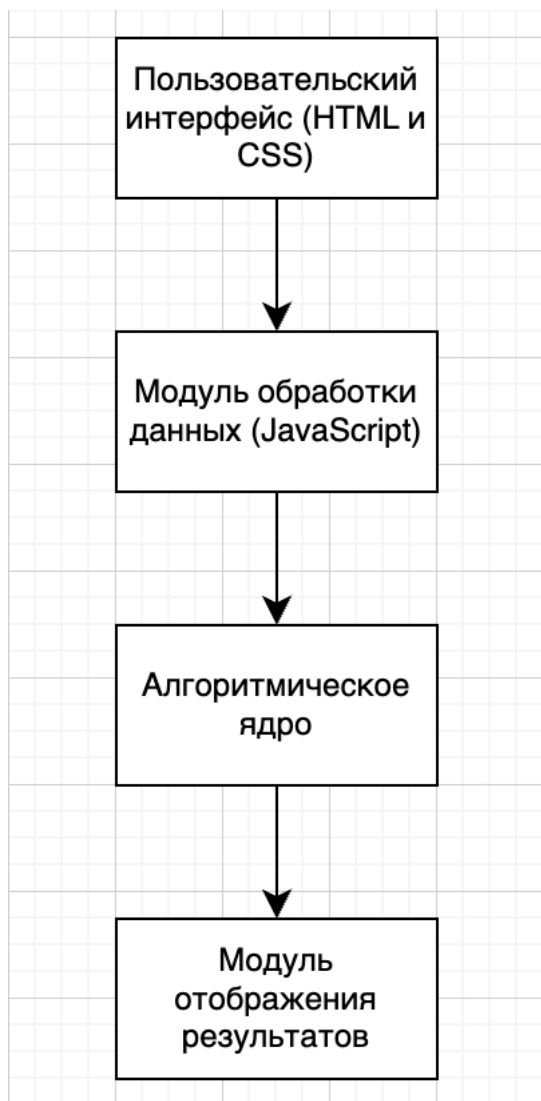


Рисунок-1. Архитектура системы

Архитектура разработанного программного приложения состоит из нескольких основных компонентов:

1. Пользовательский интерфейс

Реализован с использованием HTML и CSS. Позволяет пользователю вводить данные о продуктах и ресурсах.

2. Модуль обработки данных

Реализован на языке JavaScript. Отвечает за обработку введенных пользователем данных.

3. Алгоритмическое ядро

Реализует алгоритм полного перебора для поиска оптимального решения.

4. Модуль отображения результатов

Отображает найденное оптимальное решение, включая прибыль и количество произведенных продуктов.

Программа взаимодействует с пользователем через веб-страницу, на которой отображаются формы для ввода данных и результаты вычислений.

Также была разработана консольная версия программы на языке C++, реализующая тот же алгоритм оптимизации.

3. АЛГОРИТМ РЕШЕНИЯ ЗАДАЧИ

Для решения задачи используется алгоритм полного перебора (brute force).

Основная идея алгоритма заключается в проверке всех возможных комбинаций количества продуктов и выборе варианта с максимальной прибылью.

Алгоритм выполняет следующие действия:

1. Пользователь вводит данные о продуктах и доступных ресурсах.
2. Для каждого продукта вычисляется максимально возможное количество производства с учетом ограничений ресурсов.
3. Генерируются все возможные комбинации количества продуктов.
4. Для каждой комбинации выполняются вычисления:
 - общий расход трудовых ресурсов;
 - общий расход материалов;
 - суммарная прибыль.
5. Проверяется соблюдение ограничений ресурсов.
6. Если комбинация допустима и ее прибыль больше текущего максимума — она сохраняется как лучшее решение.
7. После завершения перебора выводится оптимальный результат.

Данный алгоритм гарантирует нахождение оптимального решения, так как рассматривает все возможные варианты.

Однако его недостатком является высокая вычислительная сложность при большом количестве продуктов.

Блок-схема алгоритма решения задачи представлена на рисунке 2.

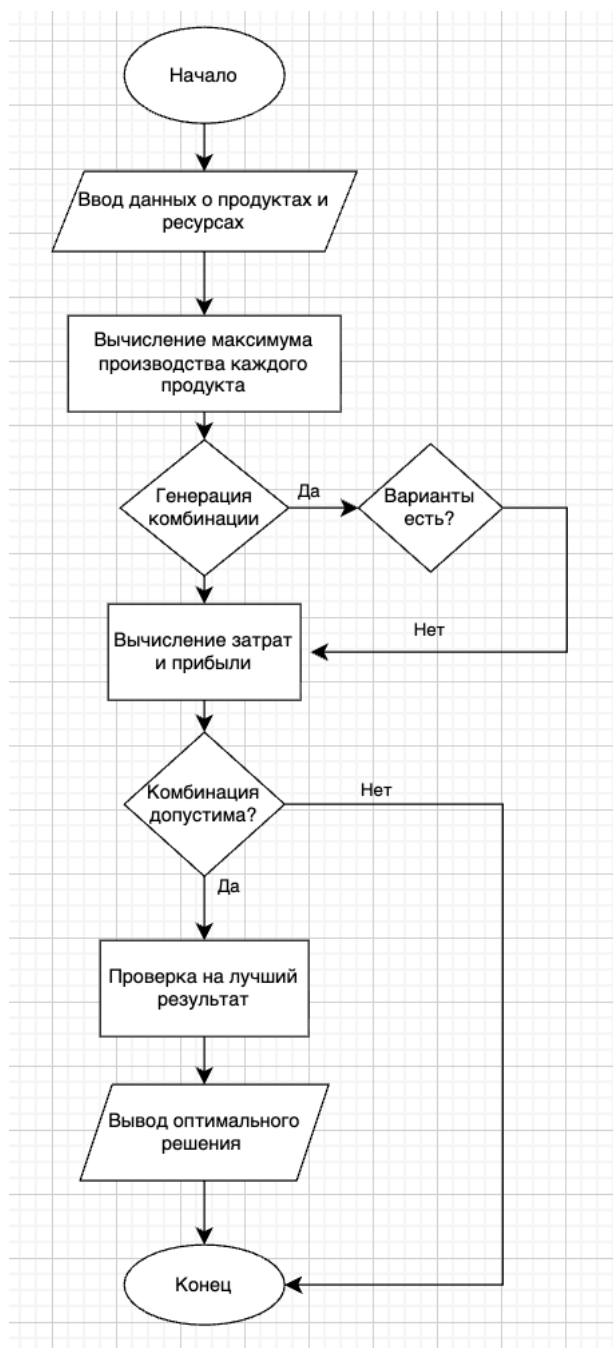


Рисунок 2 - Блок-схема

4. ИНСТРУМЕНТАРИЙ РЕШЕНИЯ ЗАДАЧИ

Для реализации проекта были использованы следующие технологии и инструменты:

- **Язык программирования C++**

Используется для создания консольной версии программы. Обеспечивает высокую производительность и эффективное управление памятью.

- **HTML**

Используется для создания структуры веб-страницы.

- **CSS**

Используется для оформления пользовательского интерфейса.

- **JavaScript**

Используется для реализации логики работы веб-приложения и алгоритма оптимизации.

- **Bootstrap**

Используется для создания адаптивного и современного интерфейса.

- **Visual Studio Code**

Используется как среда разработки программного обеспечения.

Также в проекте используется конфигурационный файл `tasks.json` для автоматической компиляции программы на C++.

5. КОД НА ЯЗЫКЕ C++

Решение задачи с помощью кода на языке C++ на основе разработанного в п. 2. алгоритма представлен в приложении А.

5.1 Алгоритм реализации веб-интерфейса на HTML и JavaScript

Для удобства пользователя была разработана веб-версия программы.

Алгоритм работы веб-приложения следующий:

1. Пользователь вводит данные о продуктах через форму на веб-странице.
2. JavaScript-код считывает введенные значения.
3. Выполняется расчет максимально возможного количества каждого продукта.
4. Запускается рекурсивный алгоритм полного перебора.
5. Для каждой комбинации рассчитывается:
 - суммарный расход ресурсов;
 - прибыль.
6. Если комбинация удовлетворяет ограничениям и дает большую прибыль — она сохраняется.
7. После завершения вычислений результат отображается на странице.

Также веб-приложение позволяет:

- динамически добавлять и удалять продукты;
- автоматически генерировать тестовые значения;
- отображать статистику работы алгоритма.

6. ТЕСТИРОВАНИЕ

Для проверки корректности работы программы было проведено тестирование с различными входными данными. Результаты тестирования алгоритма на языке C++ представлены на рисунках 3 – 5.

```
=== Оптимизация распределения ресурсов ===
Введите количество продуктов (1-10): 2

Продукт 1:
Название: станок
Труд (часы на единицу): 2
Материал (кг на единицу): 3
Прибыль за единицу: 40

Продукт 2:
Название: кран
Труд (часы на единицу): 4
Материал (кг на единицу): 5
Прибыль за единицу: 60

Общее количество часов: 100
Общее количество материала: 100
Максимум продукта станок: 33 ед.
Максимум продукта кран: 20 ед.

=== Оптимальное распределение ===
станок: 30 ед.
кран: 2 ед.
Максимальная прибыль: 1320
Всего проверок комбинаций: 714

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок 3 — Результат тестирования программы при корректных входных дан-

ных

```
=== Оптимизация распределения ресурсов ===
Введите количество продуктов (1-10): 2

Продукт 1:
Название: дерево
Труд (часы на единицу): 10
Материал (кг на единицу): 35
Прибыль за единицу: 100

Продукт 2:
Название: метал
Труд (часы на единицу): 12
Материал (кг на единицу): 48
Прибыль за единицу: 180

Общее количество часов: 3
Общее количество материала: 83
Максимум продукта дерево: 0 ед.
Максимум продукта метал: 0 ед.

=== Оптимальное распределение ===
дерево: 0 ед.
метал: 0 ед.
Максимальная прибыль: 0
Всего проверок комбинаций: 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок 4 — Результат тестирования программы при двух продуктах и недостатке ресурсов

```
=== Оптимизация распределения ресурсов ===
Введите количество продуктов (1-10): 1

Продукт 1:
Название: сено
Труд (часы на единицу): -100
Материал (кг на единицу): 100000
Прибыль за единицу: 0

Общее количество часов: 6
Общее количество материала: 0
Максимум продукт сено: 0 ед.

=== Оптимальное распределение ===
сено: 0 ед.
Максимальная прибыль: 0
Всего проверок комбинаций: 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Рисунок 5 — Результат тестирования программы при некорректных входных данных

ЗАКЛЮЧЕНИЕ

В ходе выполнения проектно-практической работы было разработано программное приложение для решения задачи оптимального распределения ресурсов.

Был реализован алгоритм полного перебора, позволяющий находить оптимальное решение при заданных ограничениях.

В процессе работы были изучены методы алгоритмизации, разработана архитектура программы и реализована программная система на языке C++ и JavaScript.

Проведенное тестирование подтвердило корректность работы разработанного приложения.

Разработанная программа может использоваться для демонстрации принципов оптимизации и изучения алгоритмов полного перебора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Страуструп Б. Язык программирования C++.
2. Шилдт Г. Полный справочник по C++.
3. MDN Web Docs — JavaScript Documentation.
4. Bootstrap Documentation.
5. Учебные материалы по дисциплине «Программирование».

ПРИЛОЖЕНИЯ

Приложение А – Листинг кода программы на С++

Приложение Б – Листинг кода программы HTML

Приложение А

Листинг кода программы на C++

```
#include <iostream>
#include <vector>
#include <string>
#include <functional>

using namespace std;

struct Product {
    string name;
    int labor;
    int material;
    int profit;
};

int main() {
    cout << "=== Оптимизация распределения ресурсов ===\n";

    int N;
    cout << "Введите количество продуктов (1-10): ";
    cin >> N;

    vector<Product> p(N);

    for (int i = 0; i < N; i++) {
        cout << "\nПродукт " << i + 1 << ":\n";
        cout << "Название: ";
        cin >> p[i].name;

        cout << "Труд (часы на единицу): ";
        cin >> p[i].labor;

        cout << "Материал (кг на единицу): ";
        cin >> p[i].material;

        cout << "Прибыль за единицу: ";
        cin >> p[i].profit;
    }

    int totalHours, totalMaterial;

    cout << "\nОбщее количество часов: ";
    cin >> totalHours;

    cout << "Общее количество материала: ";
    cin >> totalMaterial;
```

```

vector<int> best(N, 0);
vector<int> current(N, 0);
int maxProfit = 0;
long long checks = 0;

vector<int> maxCount(N);
for (int i = 0; i < N; i++) {
    int byHours = totalHours / p[i].labor;
    int byMaterial = totalMaterial / p[i].material;
    maxCount[i] = min(byHours, byMaterial);
    cout << "Максимум продукта " << p[i].name << ": " << maxCount[i] << " ед.\n";
}

function<void(int)> bruteForce = [&](int productIndex) {
    if (productIndex == N) {
        checks++;

        int hours = 0;
        int material = 0;
        int profit = 0;

        for (int i = 0; i < N; i++) {
            hours += current[i] * p[i].labor;
            material += current[i] * p[i].material;
            profit += current[i] * p[i].profit;
        }

        if (hours <= totalHours && material <= totalMaterial) {
            if (profit > maxProfit) {
                maxProfit = profit;
                best = current;
            }
        }
        return;
    }

    for (int count = 0; count <= maxCount[productIndex]; count++) {
        current[productIndex] = count;
        bruteForce(productIndex + 1);
    }
};

bruteForce(0);

cout << "\n=== Оптимальное распределение ===\n";

for (int i = 0; i < N; i++) {
    cout << p[i].name << ": " << best[i] << " ед.\n";
}

cout << "Максимальная прибыль: " << maxProfit << endl;

```

```
cout << "Всего проверок комбинаций: " << checks << endl;  
  
return 0;  
}
```

Приложение Б

Листинг кода HTML

```
<!DOCTYPE html>
<html lang="ru">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Оптимальные решения в условиях ограниченных ресурсов</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css"
rel="stylesheet">
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/boot-
strap-icons.min.css">

</head>

<body>

<div class="container">

<h2>Оптимальные решения в условиях ограниченных ресурсов</h2>

<label>Общее количество часов</label>
<input type="number" id="totalHours" value="100">

<label>Общее количество материала</label>
<input type="number" id="totalMaterial" value="100">

<h3>Продукты</h3>
<div id="productsContainer"></div>

<button onclick="addProduct()">Добавить продукт</button>
<button onclick="removeProduct()">Удалить продукт</button>

<button onclick="calculateOptimum()">Найти оптимальное решение</button>

<div id="results" style="display:none;">
<h3>Результат</h3>

<div>
Максимальная прибыль:
<span id="maxProfit"></span>
</div>

<div>
Проверено комбинаций:
<span id="checksCount"></span>
</div>
```

```

<div>
Время расчета:
<span id="calcTime"></span>
</div>

<div id="resultProducts"></div>

</div>

<div id="error" style="display:none;"></div>

</div>

<script>

let productCount = 1;
const maxProducts = 10;

window.onload = function() {
renderProducts();
};

function renderProducts() {

const container = document.getElementById('productsContainer');
container.innerHTML = "";

for (let i = 0; i < productCount; i++) {

container.innerHTML += `
<div>

<input type="text" id="name${i}" value="P${i+1}">

<input type="number" id="labor${i}" value="2">

<input type="number" id="material${i}" value="3">

<input type="number" id="profit${i}" value="40">

</div>
`;

}

}

function addProduct(){

if(productCount < maxProducts){

```

```

productCount++;
renderProducts();

}

}

function removeProduct(){

if(productCount > 1){

productCount--;
renderProducts();

}

}

function calculateOptimum(){

const totalHours = parseInt(document.getElementById('totalHours').value);
const totalMaterial = parseInt(document.getElementById('totalMaterial').value);

const products = [];

for(let i=0;i<productCount;i++){

const name = document.getElementById(`name${i}`).value;
const labor = parseInt(document.getElementById(`labor${i}`).value);
const material = parseInt(document.getElementById(`material${i}`).value);
const profit = parseInt(document.getElementById(`profit${i}`).value);

products.push({name,labor,material,profit});

}

const maxCount=[];

for(let i=0;i<productCount;i++){

const byHours=Math.floor(totalHours/products[i].labor);
const byMaterial=Math.floor(totalMaterial/products[i].material);

maxCount.push(Math.min(byHours,byMaterial));

}

let best=new Array(productCount).fill(0);
let current=new Array(productCount).fill(0);

let maxProfit=0;

```



```

let checks=0;

function bruteForce(index){

if(index===productCount){

checks++;

let hours=0;
let material=0;
let profit=0;

for(let i=0;i<productCount;i++){

hours+=current[i]*products[i].labor;
material+=current[i]*products[i].material;
profit+=current[i]*products[i].profit;

}

if(hours<=totalHours && material<=totalMaterial && profit>maxProfit){

maxProfit=profit;
best=[...current];

}

return;

}

for(let count=0;count<=maxCount[index];count++){

current[index]=count;
bruteForce(index+1);

}

}

bruteForce(0);

document.getElementById('results').style.display='block';
document.getElementById('maxProfit').innerText=maxProfit;
document.getElementById('checksCount').innerText=checks;

let resultHtml="";

for(let i=0;i<productCount;i++){

resultHtml+=products[i].name+' : '+best[i]+'<br>';

```

```
}  
document.getElementById('resultProducts').innerHTML=resultHtml;  
}  
</script>  
</body>  
</html>
```